

ITDA (잇다) 프로젝트

전체 개발 계획서

농인↔청인 소통 보조 수어 번역 웹앱
VIDEO-ONLY MODE 기반 최종 설계

팀명: ITDA (잇다) 팀

팀원: 고병진 | 인기동 | 손영란 | 손현정

목차

- 1. 프로젝트 개요
- 2. 기대효과
- 3. 현재 개발 현황 (완료 항목)
- 4. 시스템 아키텍처
- 5. 기술 스택
- 6. API 설계
- 7. 데이터 전략
- 8. 단계별 개발 계획
- 9. 미결 이슈 및 리스크
- 10. 역할 분담
- 11. 체크리스트

1. 프로젝트 개요

항목	내용
프로젝트명	ITDA (잇다) — 농인⇄청인 소통 보조 수어 번역 웹앱
팀명	ITDA (잇다) 팀
팀원	고병진 인기동 손영란 손현정
핵심 목표	한국어 텍스트/음성 입력 → 수어 영상 자동 재생으로 의사소통 지원
대상 사용자	청인 (농인에게 수어로 말하고 싶은 사람)
설계 방식	VIDEO-ONLY MODE — Supabase Storage 영상 스트리밍
직전 설계 변경	3D 아바타(Three.js) 방식 폐기 → 실제 수어 영상 직접 재생으로 전환

1-1. 변경 배경

초기 설계는 Three.js 3D 아바타 + 원본 영상을 동시에 재생하는 DUAL MODE 였습니다. 그러나 아바타 렌더링 복잡도, 리타게팅 파이프라인 유지 비용, 그리고 실제 수어 전달력의 차이를 고려하여 실제 수어사의 MP4 영상을 직접 재생하는 VIDEO-ONLY MODE 로 전환하였습니다.

2. 기대효과

구분	기대효과	세부 내용
사회적 효과	소통 장벽 해소	농인과 청인 간 언어 장벽을 낮추어 일상·공공 영역에서 원활한 의사소통 지원
사회적 효과	정보 접근성 향상	병원·관공서·상점 등에서 수어 통역사 없이도 기본적인 의사소통 가능
교육적 효과	수어 학습 지원	청인이 수어 영상을 직접 확인하며 수어를 자연스럽게 학습하는 환경 제공
기술적 효과	AI 수어 인식 고도화	카메라 기반 MediaPipe·KNN 분류기로 실시간 수어 인식 기술 발전에 기여
기술적 효과	데이터 자산 구축	1,111 개 이상의 수어 영상 데이터베이스 및 RAG 검색 엔진 축적
서비스 효과	즉각적 응답성	Supabase CDN 기반 영상 스트리밍으로 검색 후 1 초 이내 영상 제공 목표
서비스 효과	확장 가능성	80 개 시나리오 단어에서 3,695 개 수어 어휘로 확장 가능한 아키텍처 설계

3. 현재 개발 현황 (완료 항목)

3-1. 백엔드 완료

파일	기능	상태
api/main.py	FastAPI 앱, CORS, 라우터 등록	완료
api/routers/sign_language.py	POST /api/sign-language/search — RAG 수어 검색	완료
api/routers/video_proxy.py	GET /api/video/url/{keyword} — 영상 URL 반환	완료
api/routers/ws_vision.py	WS /api/ws/vision — 카메라 실시간 수어 인식	완료
api/routers/stt.py	STT 어댑터 (음성→텍스트)	완료
api/routers/collect.py	KNN 학습 데이터 수집	완료
api/services/rag_engine.py	FAISS + sentence-transformers 검색 엔진	완료
api/services/supabase_video.py	Supabase Storage REST API 연동 (aiohttp)	완료
api/services/knn_classifier.py	카메라 수어 KNN 분류기	완료
api/core/config.py	환경변수 설정 (SUPABASE_URL, SUPABASE_SERVICE_KEY)	완료
.env.example	환경변수 템플릿	완료

3-2. 프론트엔드 완료

파일	기능	상태
frontend/index.html	앱 레이아웃 — 카메라 영역 + 영상 영역 + 입력 영역	완료
frontend/js/translator.js	텍스트/STT 입력 → RAG 검색 → 영상 재생 연결 (75 줄)	완료
frontend/js/video_player.js	Supabase URL 수신 → <video> 재생 플레이어	완료
frontend/js/vision.js	카메라 MediaPipe 수어 인식	완료
frontend/js/education.js	수어 학습 모드	완료

3-3. 데이터 현황

소스	수량	상태
Supabase Storage sign_videos 버킷	1,111 개 MP4	팀원 업로드 완료
api/data/sign_video_urls.json	3,695 개 단어 + sldict URL	로컬 인덱스 완료
Supabase DB sign_lemma	1,134 개	구축 완료
Supabase DB sign_motion	1,197 개	구축 완료
Supabase DB sign_source_video	1,111 개	구축 완료

4. 시스템 아키텍처

4-1. 전체 흐름

[사용자]

```
├─ 텍스트 입력 / 음성(STT)
│   └─ translator.js
│       └─ POST /api/sign-language/search (RAG: FAISS 검색)
│           └─ keyword 반환
│               └─ GET /api/video/url/{keyword}
│                   └─ supabase_video.py
│                       └─ Supabase Storage 에 있음 → 즉시 공개 URL 반환
│                           └─ 없음 → sign_video_urls.json 조회
│                               └─ sldict 다운로드 → Supabase 업로드
│                                   └─ 공개 URL 반환
│                                       └─ video_player.js → <video> 재생
└─ 카메라 (WebSocket)
    └─ /api/ws/vision (MediaPipe → KNN 분류)
        └─ itda:rag:result 이벤트
            └─ ITDAVideoPlayer.play(keyword)
```

[서버 구성]

```
Backend : FastAPI - http://localhost:8000
Frontend : Python HTTP Server - http://localhost:3000
```

[외부 의존성]

```
Supabase Storage (sign_videos) : 영상 CDN
sldict.korean.go.kr : 원본 영상 소스 (폴백)
sign_video_urls.json : 로컬 URL 인덱스 (3,695 개)
```

4-2. 영상 서빙 상세 흐름

단계	동작	담당
1	keyword 수신 (RAG 검색 결과)	sign_language.py
2	Supabase Storage HEAD 요청 — 파일 존재 확인	supabase_video.py
3-A	[캐시 히트] 공개 URL 즉시 반환	supabase_video.py
3-B	[미스] sign_video_urls.json 에서 sldict URL 조회	supabase_video.py
4	sldict 에서 MP4 다운로드 (aiohttp, 30 초 타임아웃)	supabase_video.py
5	Supabase Storage POST 업로드 (service key 인증)	supabase_video.py
6	공개 URL 반환 → 프론트 video_player.js	video_proxy.py
7	<video> 태그 src 설정 후 재생	video_player.js

5. 기술 스택

분류	기술	버전/비고
백엔드 프레임워크	FastAPI	0.109.2
비동기 HTTP	aiohttp	3.9.3 — Supabase REST API 직접 호출
AI 검색	sentence-transformers + FAISS	RAG 기반 수어 키워드 검색
컴퓨터 비전	MediaPipe	카메라 손 랜드마크 추출
분류기	KNN (자체 구현)	수어 동작 분류
설정 관리	pydantic-settings	2.1.0 — .env 로드
스토리지	Supabase Storage	sign_videos 버킷 — MP4 CDN
데이터베이스	Supabase (PostgreSQL)	sign_lemma / sign_motion / sign_source_video
프론트엔드	Vanilla JS (ES Module)	Three.js 제거, 순수 JS
영상 재생	HTML5 <video>	MP4 스트리밍
STT	Web Speech API / 자체 어댑터	음성→텍스트 변환
서버 런타임	uvicorn	0.27.1

6. API 설계

6-1. 현재 API 목록

메서드	경로	설명	상태
GET	/api/health	서버 상태 확인	완료
POST	/api/sign-language/search	텍스트 → 수어 키워드 RAG 검색	완료
GET	/api/video/url/{keyword}	수어 영상 Supabase 공개 URL 반환	완료
WS	/api/ws/vision	카메라 실시간 수어 인식	완료
POST	/api/collect/*	KNN 학습 데이터 수집	완료

6-2. /api/video/url/{keyword} 상세

요청:

```
GET /api/video/url/안녕하세요
```

성공 응답 (200):

```
{  
  "keyword": "안녕하세요",  
}
```

```
{
  "url": "https://xxxx.supabase.co/storage/v1/object/public/sign_videos/안녕하세요.mp4"
}
```

실패 응답 (404):

```
{
  "detail": "'안녕하세요'에 해당하는 영상을 찾을 수 없습니다"
}
```

6-3. 추가 예정 API (선택)

메서드	경로	설명	우선순위
GET	/api/video/list	업로드된 영상 전체 목록 조회	낮음
POST	/api/video/batch-upload	여러 키워드 일괄 업로드	중간

7. 데이터 전략

7-1. 영상 서빙 전략

전략	설명	적용 시점
캐시 우선 (Cache-First)	Supabase 에 이미 업로드된 1,111 개 — HEAD 확인 후 즉시 URL 반환	현재 구현됨
온디맨드 업로드 (On-Demand)	없는 경우 sldict 에서 다운로드 후 Supabase 업로드 (3,695 개 커버)	현재 구현됨
사전 배치 업로드 (Pre-Batch)	sign_video_urls.json 전체 3,695 개를 미리 Supabase 에 업로드해 온디맨드 지연 제거	중기 과제

7-2. 파일명 규칙

항목	규칙	예시
Storage 버킷	sign_videos	—
파일명 생성 규칙	keyword.replace('/', '_').replace(' ', '_') + '.mp4'	안녕하세요.mp4
한글 파일명	한글 그대로 사용 (URL 인코딩은 aiohttp 가 처리)	날씨.mp4

△ 중요: 팀원이 업로드한 파일명 규칙이 위와 동일한지 반드시 확인 필요.

다른 규칙(숫자 ID, 영문 변환 등)을 사용했다면 supabase_video.py 의 _storage_path() 함수를 수정해야 함.

7-3. 시나리오 커버리지

항목	값
시나리오 대화	20 쌍 (농인↔청인)
시나리오 단어 총계	80 개
현재 커버 가능	73 개 (91.2%)
미커버 7 개	날씨, 맑다, 배려, 서툴다, 음식, 청인, 화면

7-4. 미커버 7 개 단어 처리 방안

단어	처리 방안	우선순위
날씨	sign_video_urls.json 내 '날씨' 유사어('맑음', '흐림') 매핑 또는 수동 업로드	높음
맑다	'맑음' 유사어 매핑 검토	높음
배려	직접 영상 촬영 또는 sldict 추가 검색	중간
서툴다	'어색하다' 유사어 매핑 검토	중간
음식	'식사', '밥' 유사어 매핑 검토	높음
청인	수어사전에 없을 수 있음 — '듣는 사람' 대체어 검토	낮음
화면	'스크린', '모니터' 유사어 매핑 검토	낮음

8. 단계별 개발 계획

Phase 1 — 연결 및 검증

목표: 팀원이 업로드한 영상이 실제로 재생되는 것을 end-to-end 확인

#	작업	파일	상태
1	.env 파일 생성 — SUPABASE_URL, SUPABASE_SERVICE_KEY 입력	.env	예정
2	팀원에게 파일명 규칙 확인 (예: 안녕하세요.mp4 vs word_001.mp4)	supabase_video.py	예정
3	파일명 규칙 불일치 시 _storage_path() 함수 수정	api/services/supabase_video.py	예정
4	백엔드 서버 재시작 (작업 관리자 → Python 종료 → uvicorn 재기동)	—	예정
5	<video> 태그에 autoplay 속성 추가	frontend/index.html	예정
6	end-to-end 테스트: '안녕하세요' 입력 → 영상	—	예정

	재생 확인		
--	-------	--	--

Phase 2 — 핵심 기능 완성

목표: 80 개 시나리오 단어 100% 커버 + UI 완성도 확보

#	작업	파일	상태
1	영상 로딩 스피너 추가 (로딩 중 시각 피드백)	frontend/js/video_player.js	예정
2	영상 없음(404) 시 한국어 안내 메시지 표시	frontend/js/video_player.js	예정
3	미커버 7 개 단어 유사어 매핑 추가	api/services/supabase_video.py	예정
4	카메라 인식 → 영상 재생 플로우 end-to-end 테스트	frontend/js/vision.js	예정
5	STT 음성 입력 → 영상 재생 플로우 테스트	frontend/js/translator.js	예정
6	시나리오 20 쌍 전체 대화 시뮬레이션 테스트	—	예정

Phase 3 — 안정화 및 최적화

목표: 성능 및 안정성 확보, 사전 배치 업로드 완료

#	작업	파일	상태
1	사전 배치 업로드 스크립트 작성 (sign_video_urls.json 3,695 개 전체 Supabase 업로드)	tools/batch_upload.py (신규)	예정
2	인메모리 URL 캐싱 — 동일 키워드 반복 요청 최적화	api/services/supabase_video.py	예정
3	sldict 다운로드 재시도 로직 강화 (3 회 retry, exponential backoff)	api/services/supabase_video.py	예정
4	모바일 자동재생 대응 (playsinline 속성, muted 확인)	frontend/index.html	예정
5	Supabase 미설정 시 graceful degradation (플레이스홀더 유지)	frontend/js/video_player.js	예정

Phase 4 — 완성 및 시연 준비 (최종)

목표: 시연 가능한 완성 버전 확보

#	작업	파일	상태
1	전체 시나리오 80 단어 100% 커버 확인	—	예정
2	UI/UX 최종 점검 (애니메이션, 전환 효과)	frontend/index.html	예정
3	백엔드 응답 시간 측정 및 개선 (목표: 1 초 이내)	—	예정
4	배포 환경 설정 (환경변수, CORS 정리)	api/core/config.py	예정
5	README 및 시연 가이드 작성	README.md	예정

8-5. 단계 요약

Phase	핵심 목표	핵심 산출물
Phase 1 — 연결·검증	end-to-end 영상 재생 확인	영상 재생 동작 확인
Phase 2 — 기능 완성	80 단어 커버, UI 완성	80 단어 커버, UI 완성도 확보
Phase 3 — 안정화	성능·안정성 확보	배치 업로드, 캐시, 재시도 로직
Phase 4 — 시연 준비	시연 가능 버전 완성	100% 커버, 시연 가이드

9. 미결 이슈 및 리스크

이슈	영향도	원인	해결 방법
파일명 규칙 불일치	● 높음	팀원이 업로드한 파일명이 <code>_storage_path()</code> 생성 규칙과 다를 수 있음	팀원에게 파일명 형식 확인 후 <code>_storage_path()</code> 수정
.env 미설정	● 높음	SUPABASE_URL/SUPABASE_SERVICE_KEY 없으면 모든 영상 요청 실패	.env 파일 생성 후 Supabase 대시보드에서 키 복사
백엔드 좀비 프로세스	● 중간	포트 8000 점유로 새 서버 기동 불가	작업 관리자에서 Python 프로세스 종료 후 재시작
sldict 외부 서버 불안정	● 중간	sldict.korean.go.kr 서버가 느리거나 차단될 수 있음	사전 배치 업로드로 의존성 제거 (Phase 3)
미커버 7 개 단어	● 낮음	sign_video_urls.json 에 해당 단어 영상 없음	유사어 매핑 또는 직접 영상 추가 (Phase 2)
<video> 자동재생 차단	● 낮음	브라우저 정책상 muted 없으면 autoplay 차단	autoplay + muted + playsinline 속성 추가

10. 역할 분담

역할	담당	완료 항목	예정 항목
Supabase 영상 업로드	인기동	sign_videos 버킷 MP4 1,111 개 업로드 완료	파일명 규칙 공유, 추가 업로드
백엔드 API 개발	고병진	video_proxy.py, supabase_video.py, 전체 라우터	파일명 매핑 수정, 재시도 로직,

			배치 업로드 스크립트
프론트엔드 개발	손영란	video_player.js, translator.js, index.html 영상 영역	autoplay 추가, 로딩 UI, 에러 메시지
데이터 파이프라인	손현정	sign_video_urls.json, Supabase DB 구축	미커버 7 개 단어 처리
테스트·QA	전원 공동	—	시나리오 80 단어 전체 테스트

11. 개발 완료 체크리스트

Phase 1 — 연결 검증

- ☐ .env 파일 생성 (SUPABASE_URL, SUPABASE_SERVICE_KEY)
- ☐ 팀원에게 Supabase 파일명 규칙 확인
- ☐ 파일명 규칙 불일치 시 _storage_path() 수정
- ☐ 백엔드 서버 재시작 (포트 8000)
- ☐ <video> 태그 autoplay 속성 추가
- ☐ '안녕하세요' 입력 → 영상 재생 end-to-end 확인

Phase 2 — 기능 완성

- ☐ 영상 로딩 스피너 추가
- ☐ 404 에러 시 한국어 안내 메시지
- ☐ 미커버 7 개 단어 유사어 매핑
- ☐ 카메라 인식 → 영상 재생 플로우 테스트
- ☐ STT → 영상 재생 플로우 테스트
- ☐ 시나리오 20 쌍 전체 시뮬레이션

Phase 3 — 안정화

- ☐ 사전 배치 업로드 스크립트 (3,695 개 전체)
- ☐ 인메모리 URL 캐싱
- ☐ sldict 재시도 로직 (3 회 retry)
- ☐ 모바일 자동재생 대응

- ☐ 오프라인 폴백 처리

Phase 4 — 시연 준비

- ☐ 80 단어 100% 커버 확인
- ☐ UI/UX 최종 점검
- ☐ 응답 시간 1 초 이내 확인
- ☐ README 및 시연 가이드 작성

ITDA (잇다) 팀 | 고병진 · 인기동 · 손영란 · 손현정 | VIDEO-ONLY MODE 기반